System Verilog 10

> "Software is getting slower more rapidly than hardware becomes faster."
>
> *Nicklaus Wirth [28]*

[28]: Ross (2003), *Engineering: 5 commandments*

All the parallel processing we've seen so far was enabled by individual hardware units—such as CPU or GPU cores—and their aggregation into clusters of nodes. But hardware is itself composed of large numbers of minute processing units operating in parallel. Compared to anything else we've seen, these minute units provide an extremely fine-grained parallelism. This part of the course provides an introduction to a widely-used language that is used for designing hardware.

Notes

- Hardware ultimately works by manipulating physical properties, and that manipulation usually can afford vast amounts of parallelism. Modern CPUs and GPUs compose the parallel behavior of billions of transistors. Transistors are composed to form gates and memory. Digital hardware primitives can be reduced to logic gates and memory. But even for small and simple designs, that granularity is too fine (for people) to reason directly at the level of logic gates and memory.
- *Hardware-description languages* (HDLs) are a successful and scalable abstraction for designing and reasoning about hardware. Using HDLs, designers can express the behavior and structure of hardware components and reason about their composition and resource-usage.
- Tools then take HDL code and compile it into a circuit that composes gates and memory. This circuit description can be processed further to program reconfigurable hardware,[1] or ultimately generate a description that guides the machinery that can produce an actual hardware chip.
- SystemVerilog [29] is a very widely-used HDL. In this part of the course we learn some SystemVerilog syntax, semantics, and tooling. For this course we won't use any vendor-specific tooling, but the open-source, free tooling we'll use is mature and excellent.
- We'll use the ICARUS Verilog[2] and Verilator[3] tools to map the hardware description into a program that we can run on our development machine—including on FABRIC! Running the resulting program simulates the hardware design that was expressed in SV!
- A different approach to designing hardware involves designing the hardware in a language that's used to design *software*, and then using a toolchain that converts the description for different targets. See Emu for an example [30].
- A hardware simulation can produce a *waveform* that describes how values in the hardware description change over time. These values relate to the inputs and outputs of gates, and the values stored in

1: Such as FPGAs or CPLDs.

[29]: Rich (2003), *The Evolution of SystemVerilog*

2: `https://github.com/steveicarus/iverilog`

3: `https://verilator.org/`

[30]: Sultana et al. (2017), *Emu: Rapid Prototyping of Networking Services*

memory. We can view a waveform using tools like GTKWave[4] and Surfer[5].

- In this course we'll limit ourselves to simulating hardware. But there are also great tools for *synthesizing* a hardware description into a low-level description for reconfigurable hardware or for fabrication. If you'd like to find out more, do check out Yosys.[6] To find out about this workflow, read the Yosys primer.[7] If you'd like to find out what kind of hardware was produced using this tooling, do look at open RISC-V implementations[8] and OpenTitan[9].
- Our first SV program consists of a *testbench*—a program that is intended to drive a hardware description. Our first testbench doesn't drive any hardware, it's only meant to introduce some of the structure and syntax of SV.

```
module tb;
    logic [4:0] x = 0;

    logic clk = 0;
    always #1 clk = ~clk;

    initial begin
        $dumpfile("tb_waveform.vcd");
        $dumpvars(0, tb);

        $display("Starting");

        x = 0;
        #2;
        $display("Step 0: x=%h", x);

        x = 1;
        #2;
        $display("Step 1: x=%h", x);

        x = 2;
        #2;
        $display("Step 2: x=%h", x);

        x = 3;
        #2;
        $display("Step 3: x=%h", x);

        x = 4;
        #2;
        $display("Step 4: x=%h", x);

        $display("Ending");
        $finish;
    end

endmodule
```

- We run Verilator to produce the behavioural simulation of that code in the form of a binary that we can run:

```
$ verilator --binary --trace tb.sv
```

Then we run that binary and see the following output:

```
$ obj_dir/Vtb
```

4: https://gtkwave.sourceforge.net/

5: https://surfer-project.org/

6: https://yosyshq.net/yosys/

7: https://yosyshq.readthedocs.io/projects/yosys/en/latest/appendix/primer.html

8: https://github.com/riscv/

Time

From the testbench we see an explicit account of *time* elapsing. Because of the level of detail involved, hardware design involves explicit reasoning about *time* as well as the usual "spatial" resource that programs use (i.e., memory). When designing software, time does matter of course, but usually we're abstracted from highly time-sensitive behavior—except for some microprocessors that required inserting NOPs after some operations, but that's often done by the compiler on your behalf.

```
Starting
Step 0: x=00
Step 1: x=01
Step 2: x=02
Step 3: x=03
Step 4: x=04
Ending
- tb.sv:34: Verilog $finish
```

In the first example we'll use a text-based waveform viewer that comes installed with the course materials:

```
$ ./simpleVCD/vcd waveform.vcd
```

It produces the following output:

```
11 samples / 1ps  / zoom: 1
help: q quit / ←↓↑→ scroll / ctrl←→+ zoom  / r reload
   tb      time: 0ps

    x [4:0] [ 5]:  0 0 1 1 2 2 3 3 4 4 4


                    _   _   _   _   _
        clk [ 1]: __/ \_/ \_/ \_/ \_/ \_
```

- SV can be used to produce hardware descriptions, but some code is not *synthesizable*—such as the `$display` syntax shown above. That code is only used for *simulation*.
- We unpack some key syntax: `wire`, `reg`, and `logic`. And some key values: 0, 1, Z (high-impedance — disconnected hardware, or no value available), and X (unknown)
- Note that Verilator does not support X and Z values. Change the above SV program as following:
  - Under line 2 add `logic [4:0] y;`
  - And under line 25 add `y = x;`

  And call the resulting file `tb1.sv`. First we see the output that was produced using ICARUS Verilog:

```
$ iverilog -g2005-sv -o tbicarus tb1.sv && \
  ./tbicarus && \
  ./simpleVCD/vcd tb_waveform.vcd
```

```
                    _   _   _   _   _
        clk [ 1]: __/ \_/ \_/ \_/ \_/ \_

    x [4:0] [ 5]:  0 0 1 1 2 2 3 3 4 4 4

    y [4:0] [ 5]:  X X X X X X 3 3 3 3 3
```

And compare it with the output from Verilator:

```
$ verilator --binary --trace tb1.sv && \
  obj_dir/Vtb1 && \
  ./simpleVCD/vcd tb_waveform.vcd
```

```
    x [4:0] [ 5]:  0 0 1 1 2 2 3 3 4 4 4

    y [4:0] [ 5]:  0 0 0 0 0 0 3 3 3 3 3
                    _   _   _   _   _
        clk [ 1]: __/ \_/ \_/ \_/ \_/ \_
```

- Since we want our simulation to include the X value, we'll use iverilog going forward.

Width

In the previous example, what happens if you replace `y = x;` with `y = 5'hA;`? And with `y = 5'hAA;`?

Hardware composition

Modules can be nested inside each other through instantiation.

- Next we look at some key SV datatypes. These range over 0 and 1 only: `bit`, `byte`, `shortint`, `int`, `longint`. All of these are *signed* (except for `bit`). To make them unsigned, use the **`unsigned`** keyword. There is also the **`integer`** type—this ranges over {0,1,Z,X}.
- Literals include their with and type. For example, `8'b1` and `8'b0` are 8-bit values for 0 and 1. In addition to binary (`b`), values can be encoded using other numerals—including hex and decimal. For example, in the previous example, replace `y = x;` with `y = 4'hA;`
- Code is structured into *modules* that are instantiated inside other modules—in our case, ultimately instantiated in the testbench. Through instantiation, modules are composed within containing modules. This module does nothing, but it serves to show the syntax:

```
module M (input wire W1, output wire W2);
  assign W2 = W1;
endmodule;
```

- Note the **`always`** syntax above—it is used to describe continuously-occurring behavior, usually in reaction to external changes. We'll see the syntax being used to focus on a specific set of changes of interest—through a so-called *sensitivity list*. For example, for synchronous hardware we typically include the clock signal in that list. Further, we can specify that the logic in the **`always`** block is triggered by the *edge* transition of a signal (i.e., positive or negative transition) rather than on a particular *level* of a signal's value. For example, the logic in this **`always`** block takes effect on the positive edge of the `clk` signal:

```
always @(posedge clk)
begin
  ...
end
```

The sensitivity list can be broadened to encompass all signals:

```
always @*
begin
  ...
end
```

- Next we show a stateful module:

```
/*
ubuntu@n1:~$ iverilog -g2005-sv -o tbicarus tb3.sv  && ./
    tbicarus
VCD info: dumpfile tb_waveform.vcd opened for output.
VCD warning: ignoring signals in previously scanned scope
    tb.mymodule.
Starting
Step 0: x=00, count= 1
Step 1: x=01, count= 2
Step 2: x=02, count= 3
Step 3: x=03, count= 4
Step 4: x=04, count= 5
Ending
tb3.sv:76: $finish called at 10 (1s)
*/
module  M (
  input wire clk_M,
```

```
  output wire anti_clk,
  output reg [4:0] count = 0
);

  assign anti_clk = ~clk_M;

  always @(posedge clk_M)
  begin
          count = count + 1;
  end

endmodule


module tb;
    logic [4:0] x = 0;
    logic [4:0] y;

    logic clk = 0;
    always #1 clk = ~clk;

    M mymodule (.clk_M(clk));

    initial begin
        $dumpfile("tb_waveform.vcd");
        $dumpvars(0, tb);
        $dumpvars(0, tb.mymodule);

        $display("Starting");

        x = 0;
        #2;
        $display("Step 0: x=%h, count=%d", x, mymodule.
    count);

        x = 1;
        #2;
        $display("Step 1: x=%h, count=%d", x, mymodule.
    count);

        x = 2;
        #2;
        $display("Step 2: x=%h, count=%d", x, mymodule.
    count);

        x = 3;
        y = 5'hA;
        #2;
        $display("Step 3: x=%h, count=%d", x, mymodule.
    count);

        x = 4;
        #2;
        $display("Step 4: x=%h, count=%d", x, mymodule.
    count);

        $display("Ending");
        $finish;
```

```
    end

endmodule
```

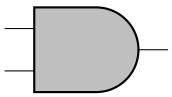
- In the previous example, we see the `assign` syntax. This is used for *continuous assignment*. Its logic can be expressed using an `always` block—specifically, the `always @*` syntax that we saw in an earlier example. Using `assign` gives us the added information that the assignment involves only combinational logic.
- Next, we update the module to implement a combinational logic function—but see how "X" dominates the value of "result" over time:

```
/*
ubuntu@n1:~$ iverilog -g2005-sv -o tbicarus tb4.sv  && ./
    tbicarus && ./simpleVCD/vcd -i=0 tb_waveform.vcd
VCD info: dumpfile tb_waveform.vcd opened for output.
VCD warning: ignoring signals in previously scanned scope
    tb.mymodule.
Starting
Step 0: x=00, result=0000x
Step 1: x=01, result=0000x
Step 2: x=02, result=0000x
Step 3: x=03, result=0000x
Step 4: x=04, result=0000x
Ending
[...]
*/
module  M (
  input wire clk,
  input wire a,
  input wire b,
  output reg [4:0] result = 0
);

  always @(posedge clk)
  begin
          result = a | b;
  end

endmodule


module tb;
    logic [4:0] x = 0;
    logic [4:0] y;

    logic clk = 0;
    always #1 clk = ~clk;

    M mymodule (.clk(clk));

    initial begin
        $dumpfile("tb_waveform.vcd");
        $dumpvars(0, tb);
        $dumpvars(0, tb.mymodule);

        $display("Starting");

        x = 0;
```

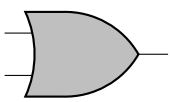
```
46          #2;
47          $display("Step 0: x=%h, result=%b", x, mymodule.
        result);
48
49          x = 1;
50          #2;
51          $display("Step 1: x=%h, result=%b", x, mymodule.
        result);
52
53          x = 2;
54          #2;
55          $display("Step 2: x=%h, result=%b", x, mymodule.
        result);
56
57          x = 3;
58          y = 5'hA;
59          #2;
60          $display("Step 3: x=%h, result=%b", x, mymodule.
        result);
61
62          x = 4;
63          #2;
64          $display("Step 4: x=%h, result=%b", x, mymodule.
        result);
65
66          $display("Ending");
67          $finish;
68      end
69
70  endmodule
```

- We review *combinational* logic.

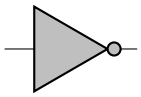
  - This involves expressions involving operators such as "and", "not", and "or". Other operators include "xor" and "xnor". We can derive logical operators from other logical operators. For example, "xnor P Q" is $(\neg P \wedge \neg Q) \vee (P \wedge Q)$.
  - Remember that *material implication* ("if P then Q", written "$P \rightarrow Q$") is logically equivalent to "$\neg P \vee Q$".
  - When we first encounter them we usually interpret them in a Boolean logic with values in $\{1, 0\}$.
  - We use *truth tables* to give the semantics of those operators. (See lecture for examples).
  - Logic operators are also denoted using symbols for *gates*. Gates abstract devices that process electrical signals. In our daily experience, gates are implemented using transistors. Symbols for gates include:
    * AND:

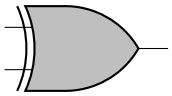
    * OR:

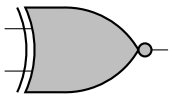
    * NOT:



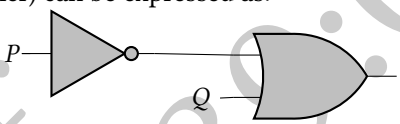
* XOR:



* XNOR:



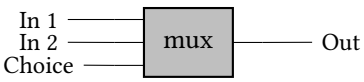
- The behavior of logic operations, and indeed of logic gates, can be expressed using other gates. For example, material implication (mentioned earlier) can be expressed as:



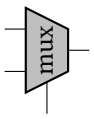
Exercise

Design a circuit (using gates other than XNOR) that implements the behavior of XNOR.

- The $(n-1)$-bit input operators/gates we mentioned above are among the 2^n operators. For example "not" is in 2^2 and "and" is in 2^3. We can find other operators of interest in this set. We can also increase n to explore operators with more inputs.
- For example, let's look at an element in 2^4 that behaves as a *multiplexer* that can choose between multiple inputs, and direct the chosen input towards the output:



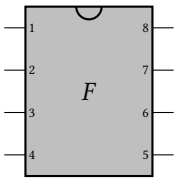
We usually use a dedicated shape for a multiplexor (mux):



Exercise

Write the truth table for "mux".

- Operators can have multiple outputs—and we can increase n to explore operators with more outputs. These operators are still within 2^n. Blurring the distinction between input and output ports, this example chip implements the abstract function F that's in 2^8:



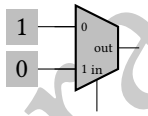
Instead of singular gates, chips integrate circuits that implement complex behaviors—both combinational and also *sequential* (stateful) behavior.

- Finally, we can add "X" to the semantics. This technically increases the space to 3^n, but in reality the space is much smaller since many functions with an X-valued input will have an X-valued output.

Exercise

Show sensible (and non-sensible) truth tables for logic gates over $\{0, 1, X\}$. (A sensible example is a 1-input gate that maps X to X, and a non-sensible example is a 1-input gate that maps 1 to X; we discuss this informal notion of "sensibility" in class.)

- We can define logical gates by using muxes, producing so-called *look-up tables* (LUTs). For example, this describes the behavior of a NOT gate:



The values shown in boxes on the left are memory cells that we can configure to define the behavior of the gate. For gates that take multiple input bits, we layer muxes.

Exercise

Define an OR gate by using muxes.

- Verilog-level *logical* operators are expressed using syntax "&&", "||" and "!".

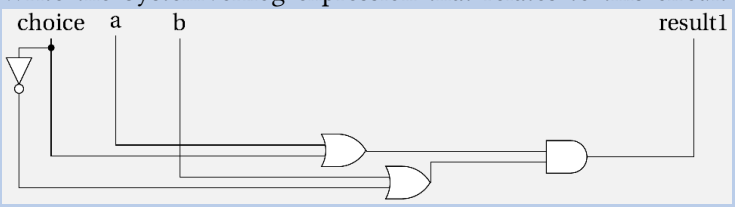
Exercise

Using SystemVerilog:

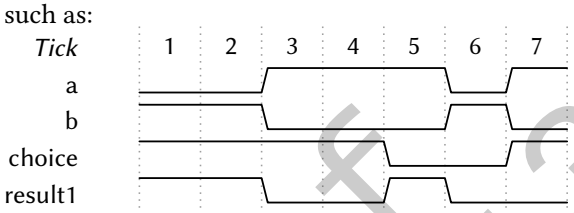
1. Define modules for different types of logic gates—e.g., AND, OR, NOT, XNOR, etc.
2. Implement a multiplexer.
3. Compose the above to implement the following by using a `always` block: a "gate" that behaves as AND or OR depending on a selection bit.
4. Express the implementation from the previous point as a single `assign` statement.

Exercise

Write the SystemVerilog expression that relates to this circuit:



- During the lectures we learnt how to interpret waveform diagrams such as:



... and relate waveforms to the testbench:

```
        count = 0;
        x = 0; // Wired to "a"
        y = 1; // Wired to "b"
        z = 1; // Wired to "choice"
        #1;

        count = 1;
        x = 0;
        y = 1;
        z = 1;
        #1;

        count = 2;
        x = 1;
        y = 0;
        z = 1;
        #1;

        count = 3;
        x = 1;
        y = 0;
        z = 1;
        #1;

        count = 4;
        x = 1;
        y = 0;
        z = 0;
        #1;

        count = 5;
        x = 0;
        y = 1;
        z = 0;
        #1;

        count = 6;
```

```
        x = 1;
        y = 0;
        z = 1;
        #1;
```

- Note that the Verilog syntax "`&`", "`|`" and "`~`" denote *bit-wise* operations. They're used on *arrays* of bits, expressed as `bit[2:0]` (for a 3-bit array in big-endian ordering) or `bit[0:2]` (for the same size array in little-endian ordering). "`^`" is a bit-wise operator denoting XOR. Notation for bit-wise negation can be composed with that of other operators to denote other operations, such as "`~|`" (NOR), "`~&`" (NAND), and "`~^`" (XNOR).
- Turning back to our earlier example in which "result" was uninitialized, the next version of our program fixes the problem with "result":

```
/*
ubuntu@n1:~$ iverilog -g2005-sv -o tbicarus tb5.sv  && ./
    tbicarus && ./simpleVCD/vcd -i=0 tb_waveform.vcd
VCD info: dumpfile tb_waveform.vcd opened for output.
Starting
Step 0: x=0, y=0, result=0
Step 1: x=0, y=1, result=1
Step 2: x=1, y=0, result=1
Step 3: x=1, y=1, result=1
Step 4: x=0, y=0, result=0
Step 5: x=x, y=z, result=x
Ending
tb5.sv:65: $finish called at 12 (1s)
[...]
*/
module  M (
  input wire clk,
  input wire a,
  input wire b,
  output reg [0:0] result = 0
);

  always @(posedge clk)
  begin
          result = a | b;
  end

endmodule


module tb;
    reg [0:0] x = 0;
    reg [0:0] y;

    logic clk = 0;
    always #1 clk = ~clk;

    M mymodule (.clk(clk), .a(x), .b(y));

    initial begin
        $dumpfile("tb_waveform.vcd");
        $dumpvars(0, tb);
        //$dumpvars(0, tb.mymodule);

```

```
44          $display("Starting");
45
46          x = 0;
47          y = 0;
48          #2;
49          $display("Step 0: x=%b, y=%b, result=%b", x, y,
        mymodule.result);
50
51          x = 0;
52          y = 1;
53          #2;
54          $display("Step 1: x=%b, y=%b, result=%b", x, y,
        mymodule.result);
55
56          x = 1;
57          y = 0;
58          #2;
59          $display("Step 2: x=%b, y=%b, result=%b", x, y,
        mymodule.result);
60
61          x = 1;
62          y = 1;
63          #2;
64          $display("Step 3: x=%b, y=%b, result=%b", x, y,
        mymodule.result);
65
66          x = 0;
67          y = 0;
68          #2;
69          $display("Step 4: x=%b, y=%b, result=%b", x, y,
        mymodule.result);
70
71          x = 'x;
72          y = 'z;
73          #2;
74          $display("Step 5: x=%b, y=%b, result=%b", x, y,
        mymodule.result);
75
76          $display("Ending");
77          $finish;
78      end
79
80  endmodule
```

- Assignments can be *non-blocking* and *blocking* (=). Non-blocking assignments are scheduled to take place concurrently. They are expressed using the symbol "<=".[10] Blocking assignments are scheduled to take place sequentially. They are expressed using the symbol "=". The difference between the two types of assignment can be seen in the context of an `always` block.
  Compare:

```
1  always @(posedge clk)
2  begin
3    a = b;
4    b = c;
5  end
```

  with:

10: This is an overloaded symbol, also meaning "less than or each to".

```
always @(posedge clk)
begin
  a = b;
end

always @(posedge clk)
begin
  b = c;
end
```

and with:

```
always @(posedge clk)
begin
  a <= b;
end

always @(posedge clk)
begin
  b <= c;
end
```

The following setup allows us to compare the two types of assignment:

```
module tb;
    reg [0:0] x;
    reg [0:0] y;
    reg [0:0] z;
    reg [3:0] count;

    logic clk = 0;
    always #1 clk = ~clk;

    always @(posedge clk)
    begin
      y = x;
      z = y;
    end

    initial begin
        $dumpfile("tb_waveform.vcd");
        $dumpvars(0, tb);

        $display("Starting");

        count = 0;
        x = 0;
        #1;

        count = 1;
        x = 0;
        #1;

        count = 2;
        x = 1;
        #1;

        count = 3;
        x = 1;
        #1;
```

```
37
38          count = 4;
39          x = 1;
40          #1;
41
42          count = 5;
43          x = 1;
44          #1;
45
46          count = 6;
47          x = 1;
48          #1;
49
50          $display("Ending");
51          $finish;
52      end
53
54  endmodule
```

- We return to our running example from earlier, and next we see a module with three outputs:

```
1  /*
2  ubuntu@n1:~$ iverilog -g2005-sv -o tbicarus tb6.sv  && ./
       tbicarus && ./simpleVCD/vcd -i=0 tb_waveform.vcd
3  VCD info: dumpfile tb_waveform.vcd opened for output.
4  Starting
5  Step 0: x=0, y=0, result1=0, result2=0
6  Step 1: x=0, y=1, result=1, result2=0
7  Step 2: x=1, y=0, result=1, result2=0
8  Step 3: x=1, y=1, result=1, result2=1
9  Step 4: x=0, y=0, result=0, result2=0
10 Step 5: x=x, y=z, result=x, result2=x
11 Ending
12 tb6.sv:70: $finish called at 12 (1s)
13 [...]
14 */
15 module  M (
16   input wire clk,
17   input wire a,
18   input wire b,
19   output reg [0:0] result1 = 0,
20   output reg [0:0] result2 = 0,
21   output wire result3
22 );
23
24   always @(posedge clk)
25   begin
26           result1 = a | b;
27           result2 = a & b;
28           //result2 <= a & result1;  // What happens if we
       try this?
29   end
30
31   assign result3 = result1 ^ result2;
32
33 endmodule
34
35
36 module tb;
```

```
    reg [0:0] x = 0;
    reg [0:0] y;

    logic clk = 0;
    always #1 clk = ~clk;

    M mymodule (.clk(clk), .a(x), .b(y));

    initial begin
        $dumpfile("tb_waveform.vcd");
        $dumpvars(0, tb);

        $display("Starting");

        x = 0;
        y = 0;
        #2;
        $display("Step 0: x=%b, y=%b, result1=%b, result2=%
    b", x, y, mymodule.result1, mymodule.result2);

        x = 0;
        y = 1;
        #2;
        $display("Step 1: x=%b, y=%b, result=%b, result2=%b
    ", x, y, mymodule.result1, mymodule.result2);

        x = 1;
        y = 0;
        #2;
        $display("Step 2: x=%b, y=%b, result=%b, result2=%b
    ", x, y, mymodule.result1, mymodule.result2);

        x = 1;
        y = 1;
        #2;
        $display("Step 3: x=%b, y=%b, result=%b, result2=%b
    ", x, y, mymodule.result1, mymodule.result2);

        x = 0;
        y = 0;
        #2;
        $display("Step 4: x=%b, y=%b, result=%b, result2=%b
    ", x, y, mymodule.result1, mymodule.result2);

        x = 'x;
        y = 'z;
        #2;
        $display("Step 5: x=%b, y=%b, result=%b, result2=%b
    ", x, y, mymodule.result1, mymodule.result2);

        $display("Ending");
        $finish;
    end

endmodule
```

- Logic starts out with value X and we usually use a *reset* signal to initialize. While we can check for equality between $\{0, 1\}$ values by using "==" (or conversely "`!=`"). That operator cannot check for equality be-

tween {0, 1, X}. For that, we use the "===" (conversely "!==") (but note that that comparison operator is *unsynthesizable*). Other comparison operators are familiar from other languages: ">", "<", "<=", etc.

- Arithmetical operators are expressed using the familiar syntax: "+", "-", "*", and "/", and the shift operators ">>" and ">>>" (and their left-pointing counterparts). Note that ">>" inserts zeroes while ">>>" will insert zeroes or ones depending on the value of the leading bit (which indicates the "sign" of an integer).

Considerations for operators

SystemVerilog provides syntax for other operators such as "%" (mod) "**" (exponentiation) but note that some operators might not synthesize, or synthesize inefficiently.

- Next we take a closer look at *sequential* logic. This extends combinaional logic with memory. Two common abstractions for a single bit of such memory consist of *latches* and *flip-flops*. The first is level-triggered while the second is edge-triggered—usually on the edge of a clock signal.
- We saw this example of a counter:

```
module Counter (
  input logic clk,
  input logic reset,
  output reg [2:0] count
        );

  always @(posedge clk) // reset or
  begin
    if (reset) count = 0;
    else count = count + 1;
  end

endmodule;

module tb;
  logic clk = 0;
  logic reset;
  logic [2:0] step;
  logic [2:0] count_output;

  Counter counter(.clk(clk), .reset(reset), .count(
    count_output));

  always #1 clk = ~clk;
  initial begin
    $dumpfile("tb_waveform.vcd");
    $dumpvars(0, tb);
    $display("Starting");

    #2
    step = 0;
    reset = 1;
    $display("Update: step=%d", step);

```

```
    #2
    step = 1;
    reset = 0;
    $display("Update: step=%d", step);

    #2
    step = 2;
    $display("Update: step=%d", step);

    #2
    step = 3;
    $display("Update: step=%d", step);

    #2
    step = 4;
    $display("Update: step=%d", step);

    #2
    step = 5;
    reset = 1;
    $display("Update: step=%d", step);
    #2
    step = 6;
    reset = 0;
    $display("Update: step=%d", step);
    #2
    step = 7;
    $display("Update: step=%d", step);
    #2
    step = 0;
    $display("Update: step=%d", step);
    #2
    step = 1;
    $display("Update: step=%d", step);

    $display("Ending");
    $finish;
  end
endmodule
```

Exercise

Change the behavior of the counter to use explicit signals to *increment* and *decrement* the counter.

- For further examples of hardware component descriptions, see the code that forms part of the Emu prototype.[11]
- There is so much more that can be learnt about SystemVerilog and hardware design. If this interests you, there's a wealth of resources online that can help complement the brief introduction we had in this introductory course. There are also relatively inexpensive FPGA boards that can be used to obtain more practical experience. Within a university, more extensive development boards can usually be found among the resources of a group carrying out performance-oriented research—reach out to faculty to find out about what they're working on. For an example of recent work, see our demo from SC24,[12] related

11: https://github.com/NaaS/emu-live/tree/master/lib

12: https://sc24.supercomputing.org/wp-content/uploads/2025/02/nre009.pdf

to the TransparNet project.[13]

13: https://transparnet.cs.iit.edu/